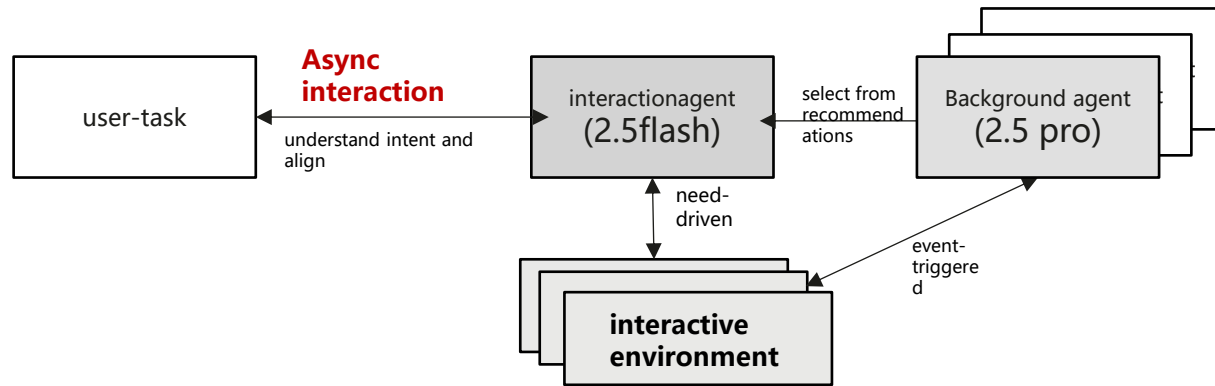


Low-latency agent

Agent form factor: from collaboration to asynchronous autonomous task completion; interaction shifts from passive Q&A to proactive recommendation and user selection

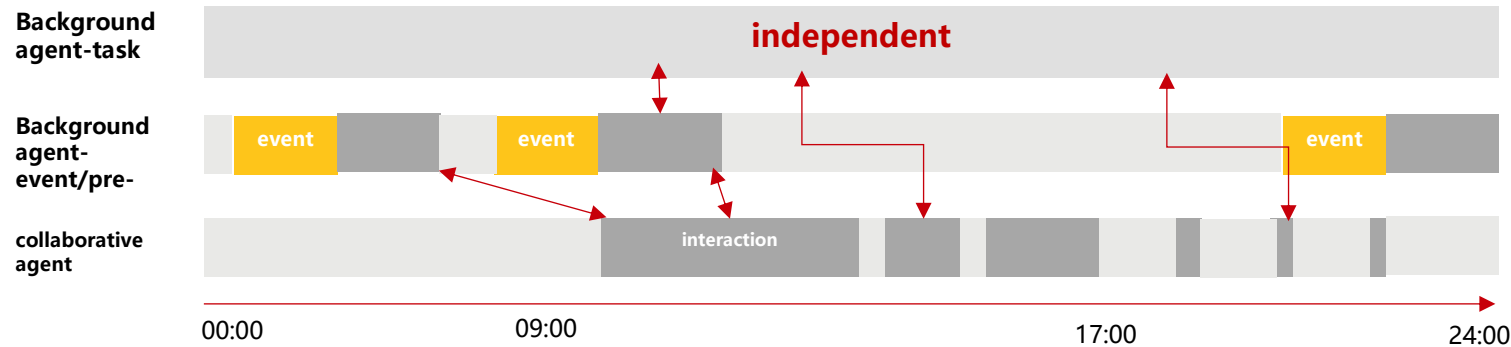


Agents can independently complete longer tasks

Subtrend 1: human time is finite and non-scalable -> agents can work 24/7

Subtrend 2: human interaction frequency is limited -> agents need to work more autonomously

Subtrend 3: human capability is limited -> **agents need to complete more complex tasks**



Front-end agent

- delivers an extreme omni-modal interaction experience
- low latency: interaction is latency-sensitive
- **user**
- interaction: taskfeedback

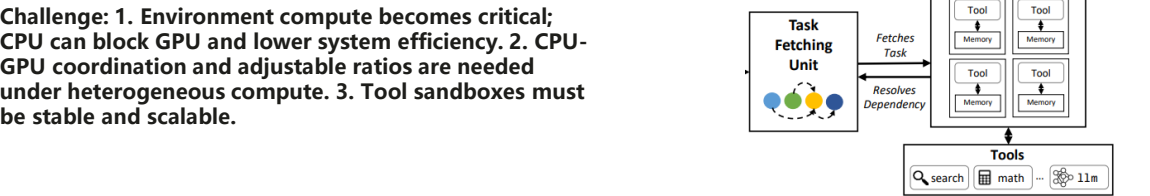
Back-end agent

- **Stateful**: has long-term memory, continuously stores experience and self-iterates
- autonomous task completion: hour-, day- and month-scale tasks
- event-driven task-triggered
- pre-reasoning: can pre-process high-frequency/common scenarios/and information

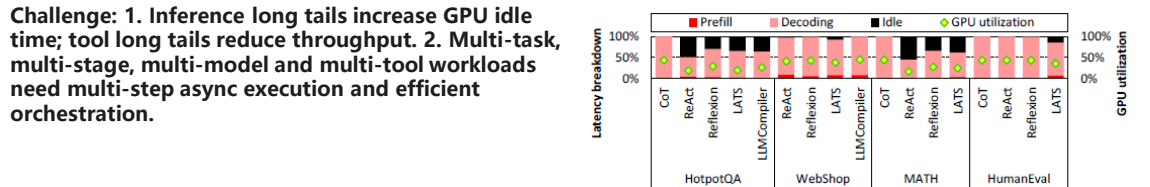
Biggest agent-system challenge: heterogeneous environment compute, long-horizon async orchestration, multi-tier experience storage, and low-latency high-concurrency multi-agent serving

New changes + challenges

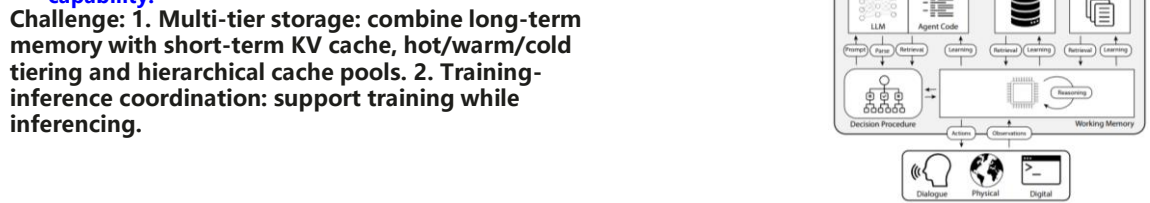
Environment scaling: joint exploration across complex environments. One Agent will handle control, information retrieval, coding and hundreds/thousands of MCP ecosystem tools.



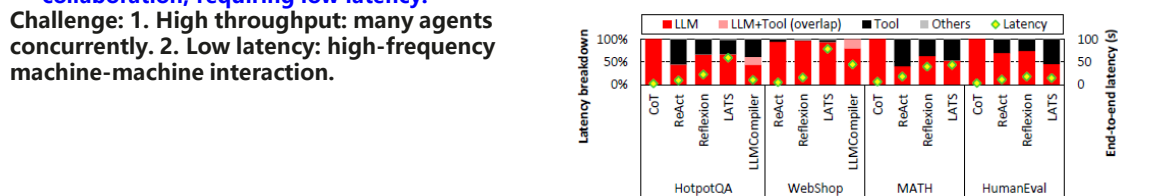
Trajectory scaling: multi-turn decision-making and reasoning; task duration can reach day/week scale.



Experience scaling: accumulate experience through environment interaction and learn to improve capability.

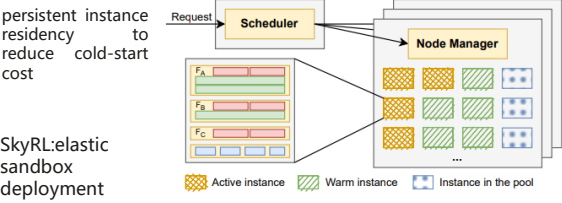
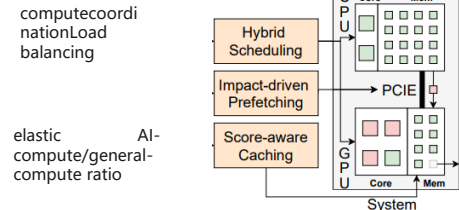


Agent scaling: many agents deployed in parallel, with high-frequency interaction and collaboration, requiring low latency.

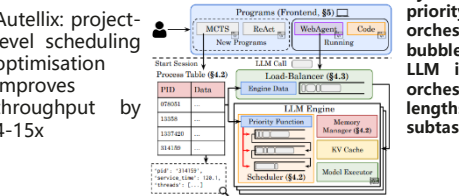


Industry approach

Heterogeneous compute coordinations scheduling

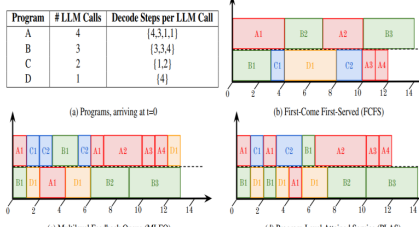


Multi-tier async hybrid inference

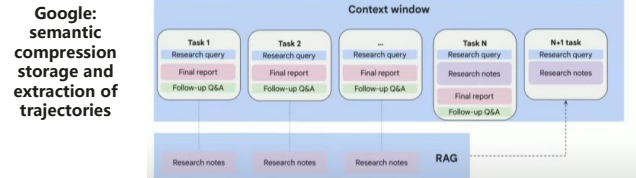


Dynamic task dispatch: dynamically split tasks based on priority and dependencies, and orchestrate efficiently to reduce bubbles

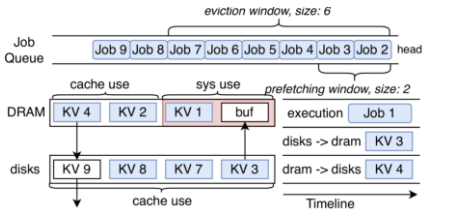
LLM inference and tool calls: orchestrate stages with different lengths and counts within



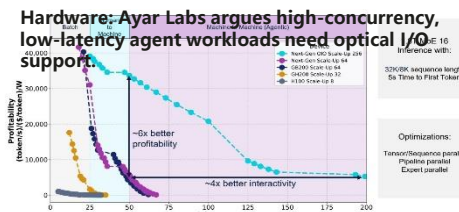
Pooled sharing of experience and memory



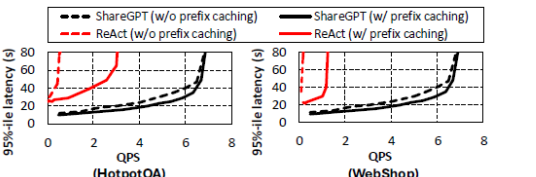
tiered storage of experience data



software-hardware co-design



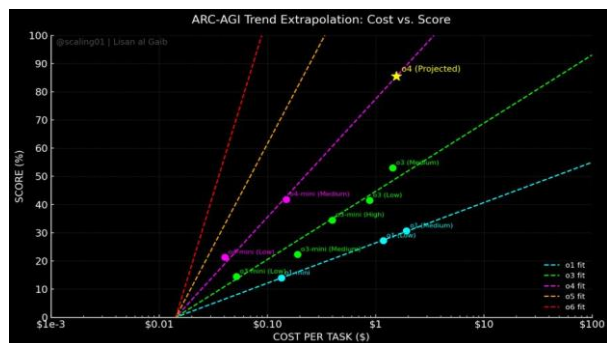
Algorithm: efficient prefix caching greatly improves QPS



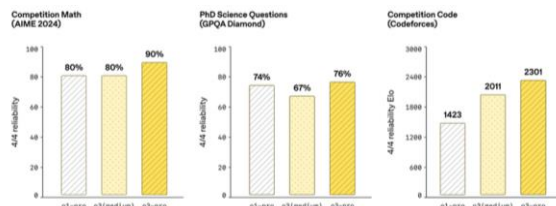
End-to-end agent experience: accuracy, robustness, efficiency and latency

Agent experience target = task completion x reliable completion x low-cost completion x low-latency experience

Accuracy keeps pushing higher

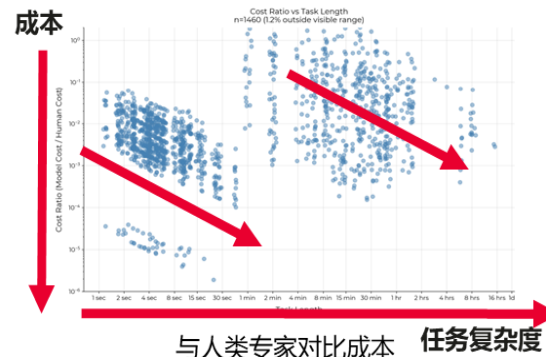


Task robustness (4/4 Reliability)

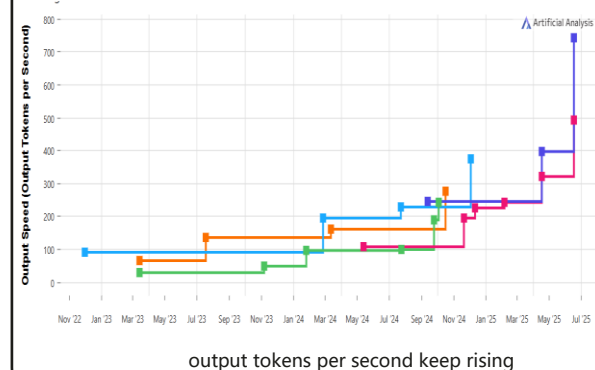


Use the 4/4 Reliability metric: a model is counted as successful only if all four attempts are correct

Improve task efficiency

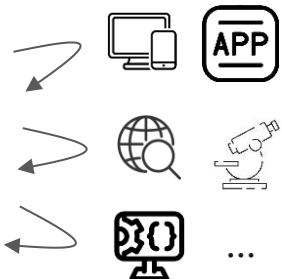


Model generation speed keeps improving



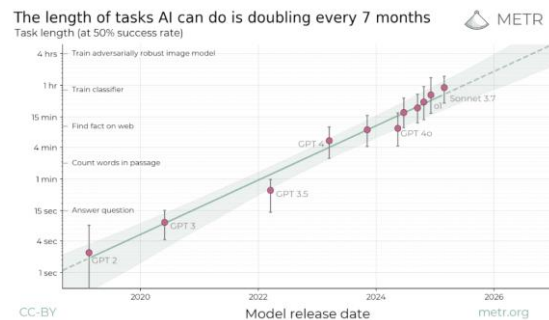
Future agent deployment trend: real complex environments x independent long tasks x multi-agent deployment x async interaction

multiple complex digital environments



multi-complex environment, Agent control, information retrieval, coding, etc. multi-MCP tool = Agent: GPT5? Gemini 3?

independent long tasks

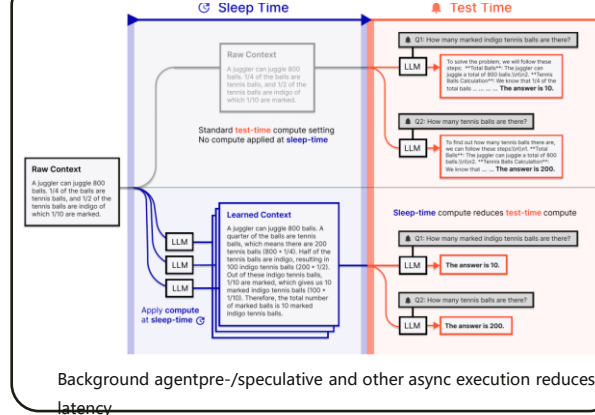


multi-agent deployment

tokens fast. In our data, agents typically use about 4× more tokens than chat interactions, and multi-agent systems use about 15× more tokens than chats. For economic viability, multi-agent systems

Anthropic: agent multi-agents system token chat multi-415

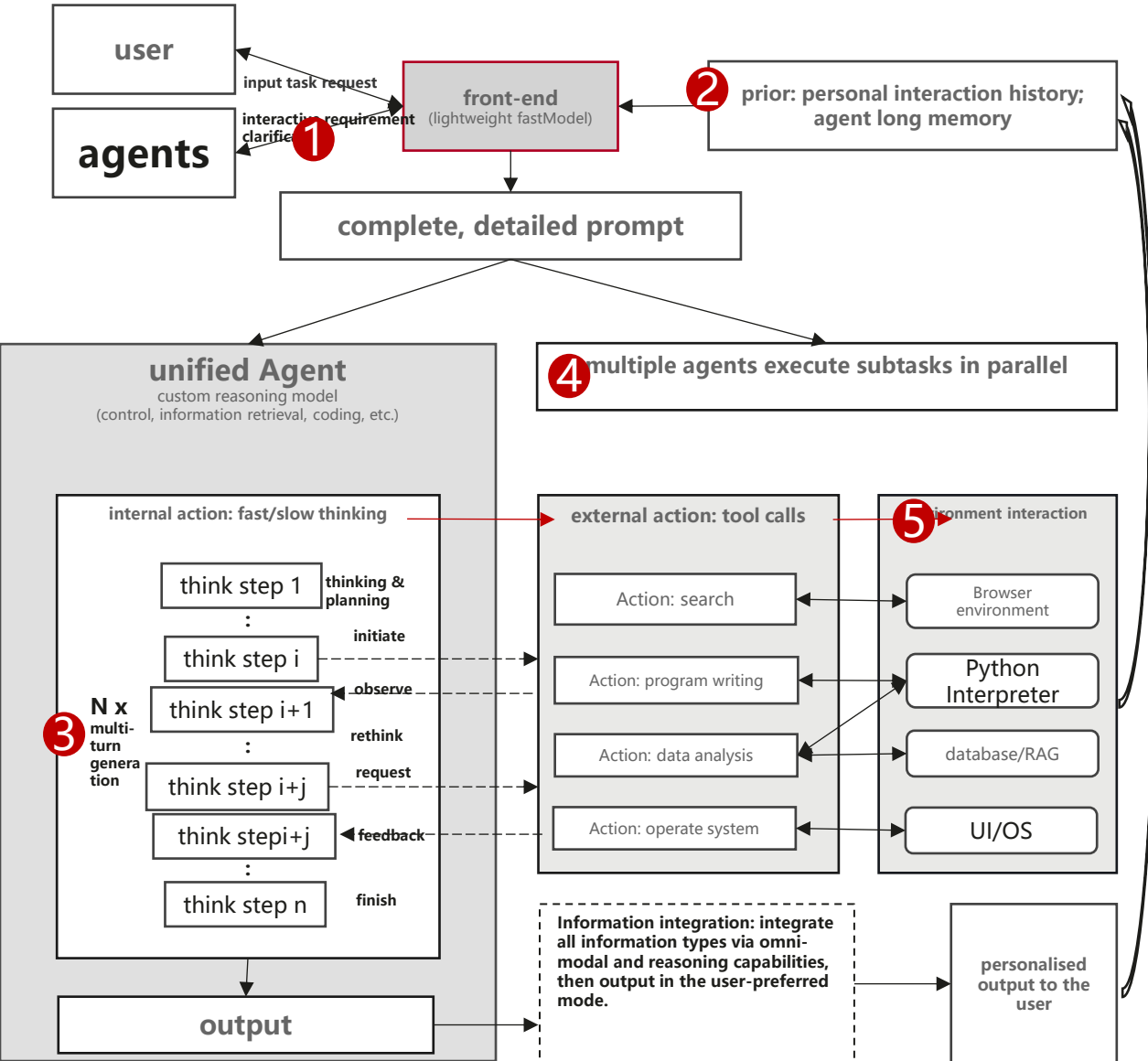
Async interaction



Agent architecture: 80% general-purpose inference acceleration; 20% scenario-specific modular composition

agent model based on the ReACT paradigm

Unique features from reasoning models to agentic models



1 Agent machine-machine interaction: high-frequency, 24/7

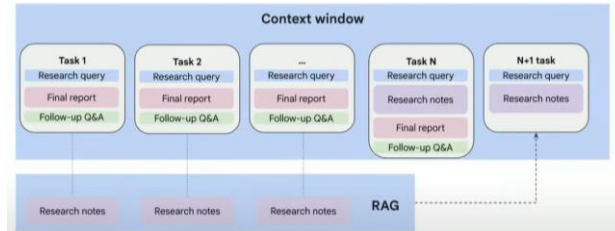
- system supports ultra-high QPS
- **low-latency inference and tool calls support high-frequency inter-agent interaction**
- highly reliable inference system can run continuously

2 life-scale historical trajectories + inter-agent interaction history

- combine long-term memory storage with short-term KV cache, hot/cold tiering of storage resources, hierarchical cache resource pools
- full logging of agent execution and interaction, log everything

3 multi-turn ultra-long trajectories: hours, days, weeks

- Latency contradiction
- Long-context management



4 multi-agent deployment: hundreds/thousands in parallel

- multi-agent orchestration
- fast deployment and scalability

5 Environment: reliable, stable, accelerated execution

- tools/environments are strongly consistent across time, with the same result each run
- runtime environment is stable and errors do not accumulate over time
- accelerated execution to avoid tool I/O bounds
- feedback environment tool

Overall trend in agent inference: more compute, more memory, longer latency, elastic scheduling

- Agent-inference efficiency directly affects RL throughput and deployed-agent ROI/user experience. Full-stack software-hardware optimisation is needed for agent workloads.

More compute: long trajectories x many tools x many agents sharply increase inference-resource demand

- Anthropic: single-agent and multi-agent systems consume 4x and 15x more tokens than chatbots, respectively
- multi-environment interaction; multi-tool deployment

Prediction: inference compute scale could grow 1000x
 Compute-utilisation challenge: how to orchestrate subtasks to avoid bubbles, sense load in real time, and schedule the most suitable compute resources

More memory: multi-turn long trajectories x personalisation expand KV cache

- Multi-turn long trajectories: agents and tools are stateful, and task quality depends on long memory
- Agent personalisation means memory cost grows with interaction duration

Prediction: memory demand will keep rising and constrain ultra-long-task performance

Memory optimisation challenge: how to use pooled memory, improve prefix-cache hits, and optimise/compress KV cache

Longer latency: tool I/O creates a new bottleneck

- Combining fast/slow thinking with tool calls increases end-to-end latency
- Agent-interaction+ - environment interaction- interaction high, latency low

Prediction: low-latency machine-machine interaction conflicts with longer tool-call latency

End-to-end latency challenge: how to run inference and tools asynchronously in parallel; parallelise multiple tools; and balance LLM inference latency against tool-call latency

Elastic scheduling: dynamic task orchestration and reliable inference

- Subtasks in long tasks have different priorities
- Agent tasks vary widely, dynamically calling many agents/tools/environments, so compute-resource needs fluctuate and require elastic scaling
- The inference system must be stable and recover quickly

Prediction: systems must dynamically orchestrate many agents/tools and elastic compute resources to meet SLOs

Challenge: how to accelerate CPUs, dynamically tune CPU:GPU ratios, and coordinate GPU-CPU execution

Implications for agent optimisation:

Learn when to generate versus use tools: output efficiently based on task needs and choose the better cost, latency and price-performance path

Parallel and speculative tool-calling algorithms reduce latency: borrow from speculative decoding, predict tool and LLM inference load, and speculatively use tools

Agentic AI will challenge compute systems on heterogeneous compute, multi-step async execution and dynamic

scheduling

- Environment exploration, long trajectories, multi-agent execution and self-evolution make Agentic RL training more complex; as overall RL compute scales faster, improving compute-resource utilisation becomes mandatory
- Environment compute will become a critical layer for both training and deployment

Environment exploration scaling



More compute: long trajectories x many tools x many agents sharply increase resource demand

Anthropic: single-agent and multi-agent systems consume 4x and 15x more tokens than chatbots, respectively

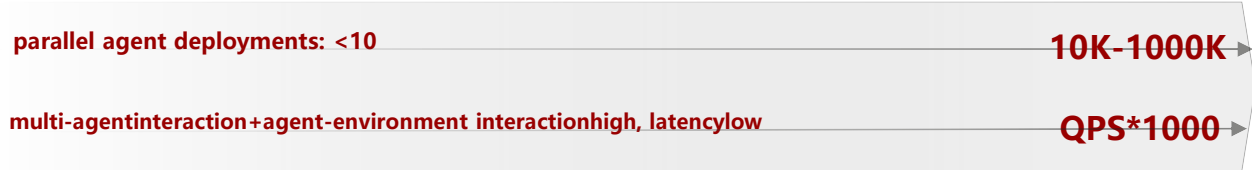
Trajectory scaling



More memory: multi-turn long trajectories x personalisation expand KV cache

KAIST: tool-using agents have 3-5.4x more memory overhead than CoT

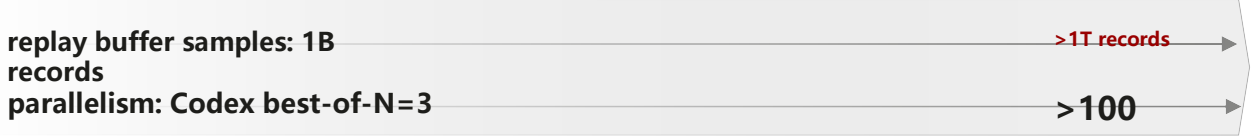
Agents Scaling



latency: tool/Onw, latency, largel long-tail orchestration bubbles, low

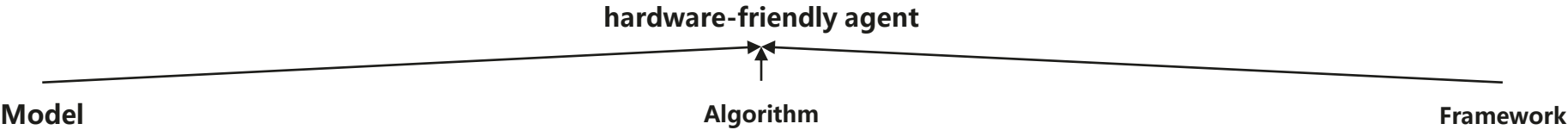
ReACT: tool latency accounts for >50% of total latency

experience Scaling



challenge1-computing:-interaction-feedback-trainingdeployment; CPU acceleration and scaling; CPU-GPUcoordination
Challenge 2 - multi-tier storage: combine long-term memory with short-term KV cache; hot/warm/cold storage tiering; hierarchical cache resource pools
Challenge 3 - async orchestration: current max is 4 steps; need to support 10+ steps, pipeline orchestration, elastic resources and load balancing

computechallengefeedbackModelAlgorithm, system-friendly agent



- 1. Efficient inference, compute-friendly: medium-scale models and high-sparsity MoE models suited for inference scaling
- 2. Long-context, memory-friendly: linear or hybrid attention such as Lightning Attention

- 1. Async-training RL algorithms reduce accuracy loss: Meta LlamaRL AIPO and Ant AREAL async PPO
- 2. speculative and parallel tool-calling algorithms reduce latency by borrowing from speculative decoding

- 1. Use the same environment for training and production deployment: OpenAI Codex learns in real environments and builds smaller, better-tested modules
- 2. Learn when to generate versus use tools: choose the better cost, latency and price-performance path by scenario

inference-token cost 15x

Anthropic: agentmulti-agentsystemtokenchatmulti-415

Challenge: inference compute demand could grow 1000x, requiring larger inference clusters

High-throughput RL needs inference-interaction-training decoupling and async execution

- RL accuracy declines as data becomes stale
- currently supports at most 4 async steps

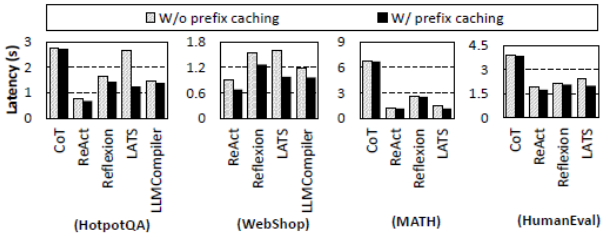
Challenge: long latency from multimodal generation and tool calls requires more async depth, around 10 steps

Agents need isolated and stateful environments

- environments can scale independently

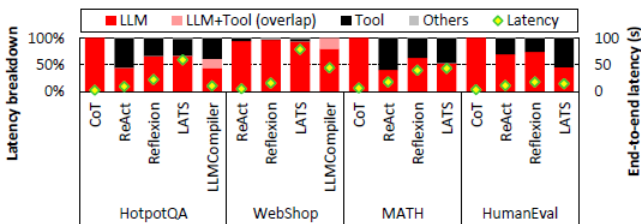
Challenge: Environment compute must remain stable and preserve output robustness

accumulated long context causes high latency



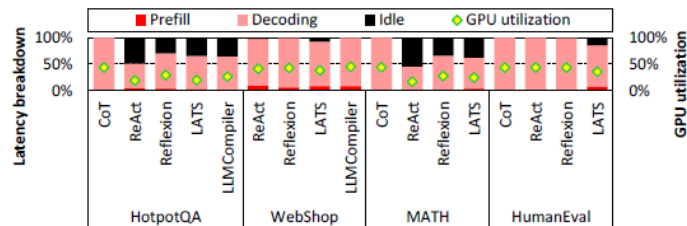
Challenge: repeated trajectories become the next-round context, creating large KV-cache cost

Tool calls materially increase end-to-end latency



Challenge: tool-call I/O latency can account for up to 50%

Inference generation-bound + tool I/O-bound

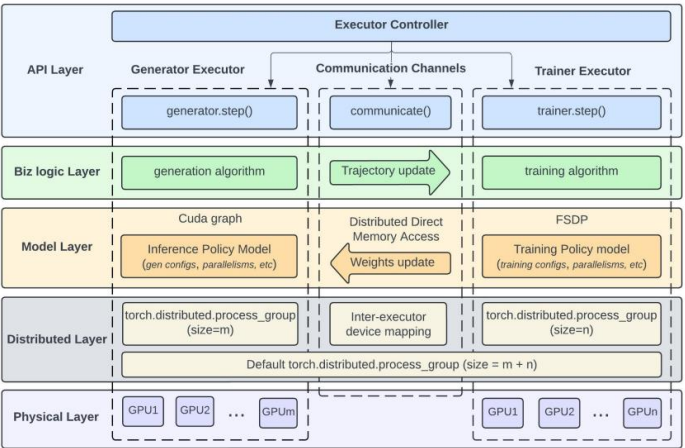


Challenge: serial inference and tool execution -> GPU idle time

Industry update: RL infrastructure iteration is accelerating and entering production to support next-generation model training

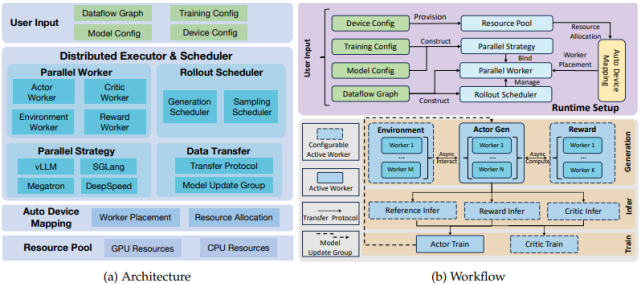
6.2 Meta LlamaRL: distributed async RL, 10.7x acceleration on a 405B policy

AIPO, an RL algorithm optimised for async training, reduces the impact of off-policy data on training accuracy



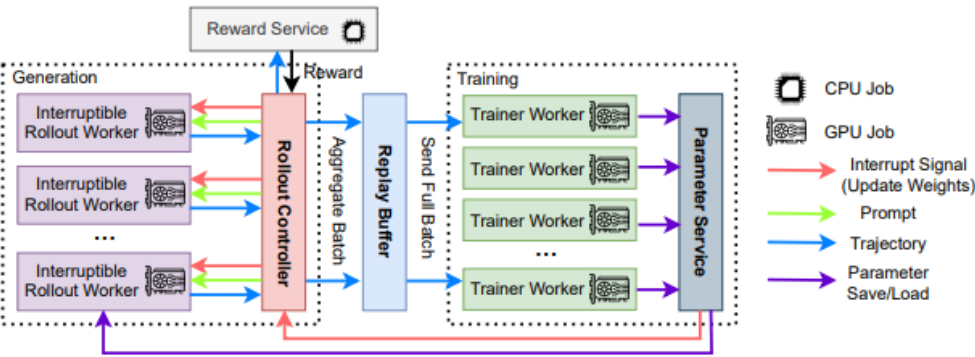
6.6 Alibaba ROLL: supports environment workers and reward workers for Agentic RL needs

large agentic RL: task multi-agent-environment interaction;
scalable environment scaling High throughput rollout;
environment actor as ncp parallel, GPU



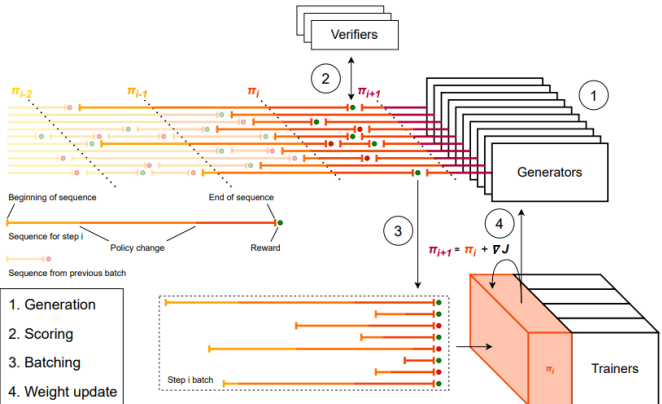
6.4 Ant AREAL: decoupled training/inference async RL, delivering 2.77x

Fully async; controls data staleness; dynamic microbatching; scalable CPU deployment for complex reward services



6.10 Magistral: decoupled training/inference deployment for async RL

Batch size can vary dynamically as generation length changes during post-training RL



Task understanding -> compute DAG -> task orchestration -> tool calls -> compute orchestration

Work 1: Autonomous Deep Agent

Hierarchical Task DAG (HTDAG)

framework, Autonomous API & Tool

Creation (AATC), Autonomous

Feedback Learning (AFL)

Work 2: An LLM Compiler for Parallel Function Calling

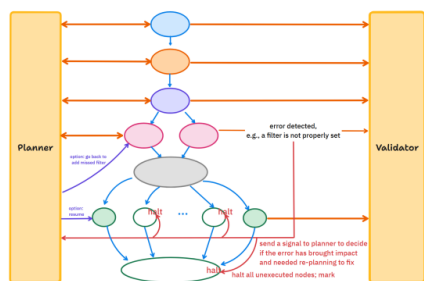
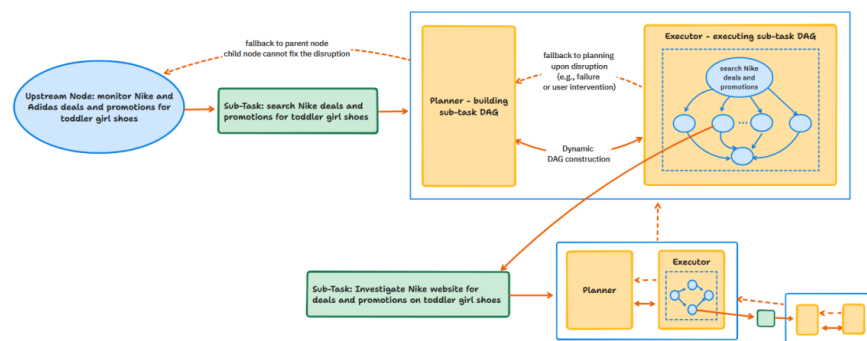
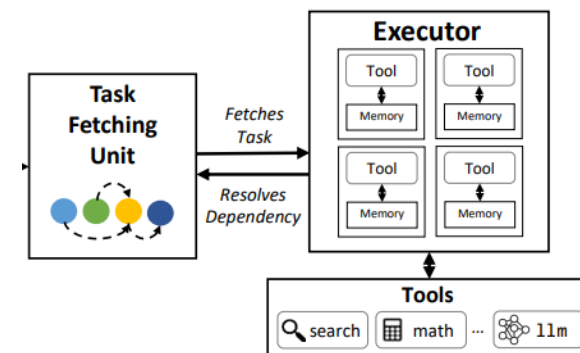
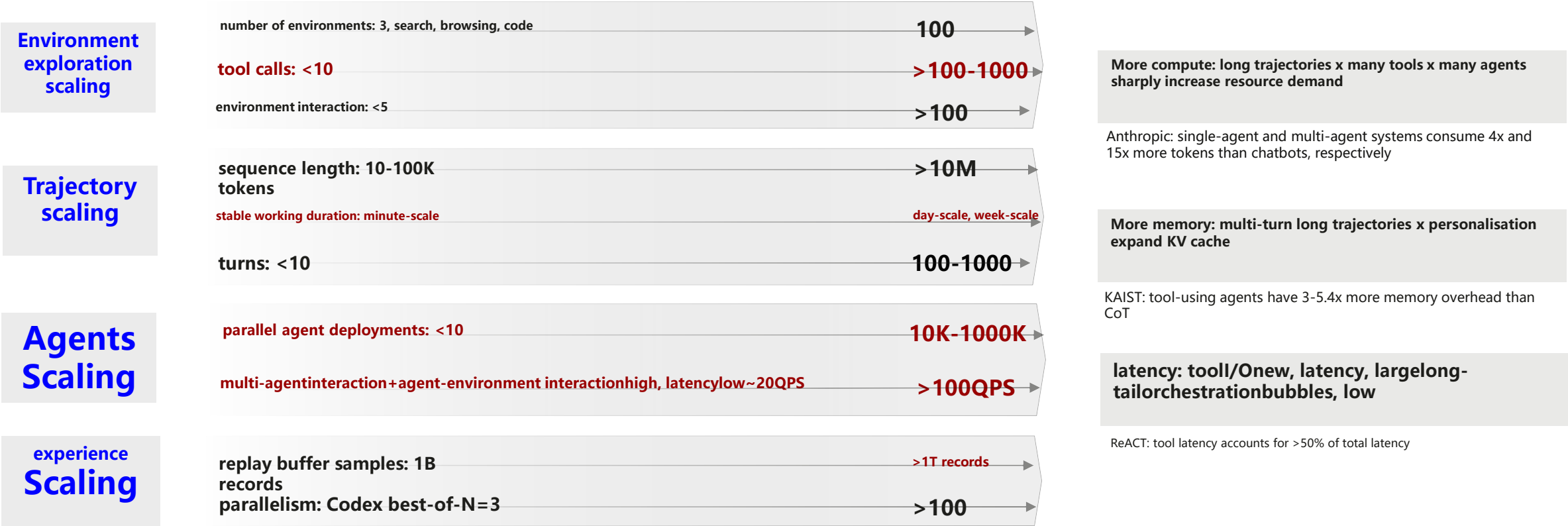


Figure 7: Validation architecture showing the interaction between the planner, task DAG and Validator component. We reuse the example from Figure 3 (b-1) here. The Validator monitors execution outcomes at each step as a separate parallel flow. Upon error detection, the Validator halts unexecuted nodes in the task DAG and signals the planner for re-planning. The planner then assesses the error impact: it may proceed if the error is benign (e.g., missed setting "sales" filter but all user-chosen items at the confirmation step were sales items), or it may rescind currently unexecuted nodes and correct the issue (e.g., add missing filter, re-confirm with the user, and restart the item investigation).



Agentic AI materially increases compute, memory and latency demands, challenging compute systems on heterogeneous compute, multi-step async execution

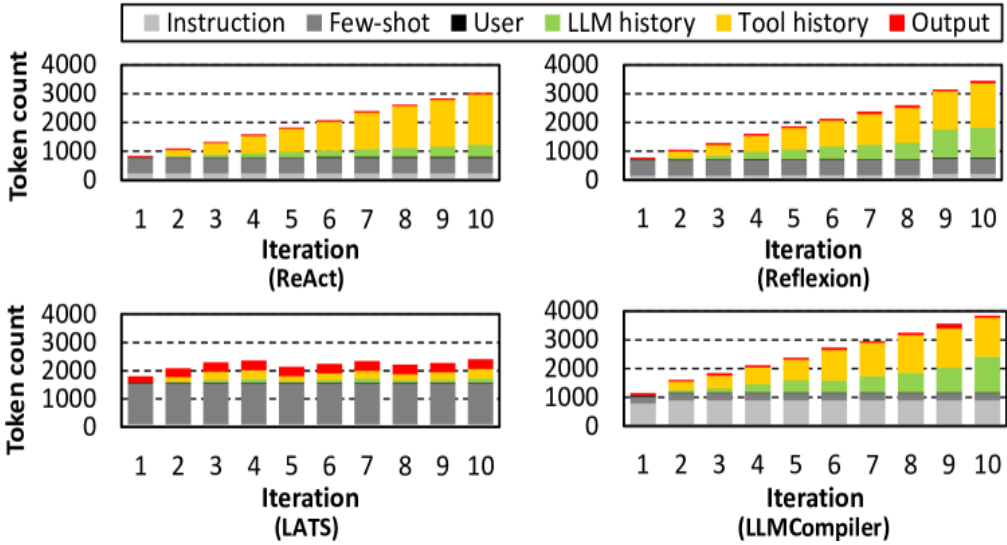
- Environment exploration, long trajectories, multi-agent execution and self-evolution make Agentic RL training more complex; as overall RL compute scales faster, improving compute-resource utilisation becomes mandatory
- and dynamic scheduling



challenge1-computing:-interaction-feedback-trainingdeployment; CPU acceleration and scaling; CPU-GPUcoordination
Challenge 2 - multi-step async: current max is 4 steps; need to support 10 steps and pipeline orchestration
Challenge 3 - dynamic workload: elastic resources, load balancing and high system reliability

userReActhighdata, Model

Application scenario	Token length before tool call	Token length after tool call
Arithmetic computation	~100 tokens	~160–180 tokens
Programming execution	~130 tokens	~250–300 tokens
Search Q&A	~150 tokens	~350–500 tokens
Multi-tool chained calls	~200 tokens	~600–1000 tokens



Scenario type	Self-evolution training direction	Input token range	Output token range	Total token range	Average ReAct hops
Mathematical reasoning	strengthen symbolic computation	120-350	80-200	200-550	3~50
	multi-step derivation capability				
tool	improve API parameter accuracy	180-600	40-150	220-750	2~6
	error recovery capability				
Knowledge Q&A	optimise fact checking	250-1200	150-500	400-1700	4~10
	multi-source information fusion				
Code generation	improve API compatibility	300-900	200-800	500-1700	4~70
	exception-handling completeness				

Multi-task: from single-digit subtasks to >10 subtasks
Multi-environment: from a few simulated environments to real environments
Multi-tool: from todays 3 main tools to 100+ digital tools
Multi-turn: from todays 1-3 turns to tens or hundreds of turns
Multi-reward: dense rewards from multi-turn, multi-level, diverse PRMs

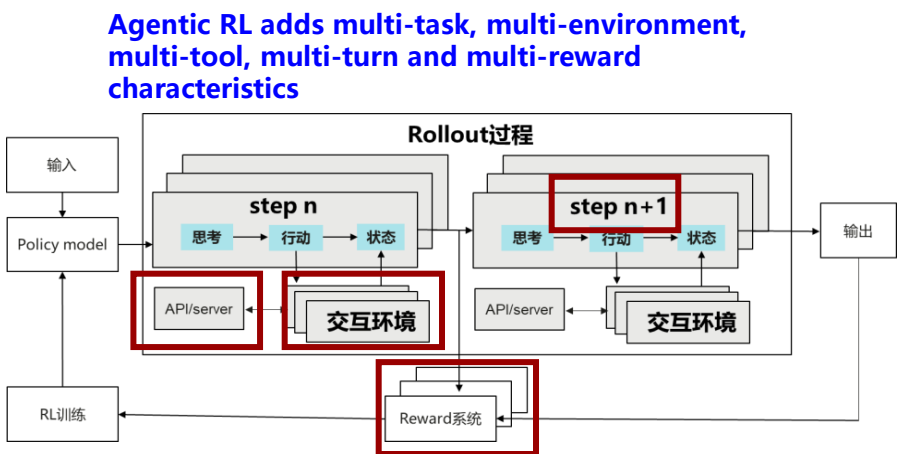
Parameter	Category	Selection	\$ Cost	Power	Latency	Quality
GPU Generation	Hardware Type	Newer	Higher	Higher	Lower/No Change	No Change
CPU vs GPU	Hardware Type	CPU	Lower	Lower	Lower	No Change
Task Parallelism	Resource Amount	More Fan Out	Higher	Higher	Lower	No Change
Execution Paths	Resource Amount	More Paths	Higher	Higher	Higher/No Change	Higher/No Change
Model/Tool	Agent Implementation	More Parameters	Higher	Higher	Higher	Higher/No Change

Agentic RL training: environment compute becomes a critical layer; GPU-CPU coordination and sandbox deployment

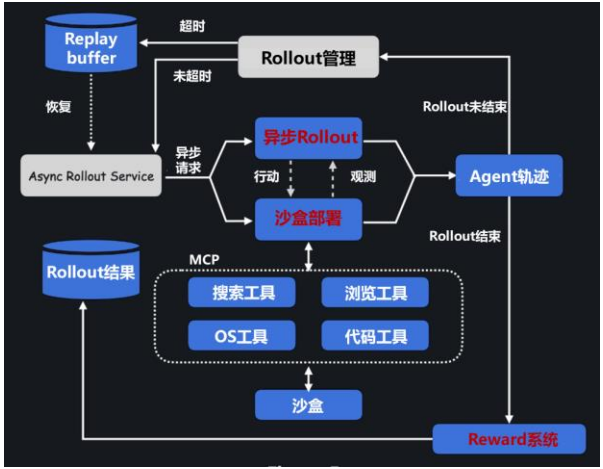
Environment exploration, long trajectories, multi-agent execution and self-evolution make Agentic RL training more complex; as overall RL compute scales faster, improving compute-resource utilisation becomes mandatory

Environment compute will become a critical layer for both training and deployment

are needed to improve end-to-end throughput



Core tool-calling components: async execution, sandbox deployment and reward



environment compute

Challenge: environment deployment needs CPUs; high-concurrency serial execution can block GPUs and reduce system efficiency

Challenge: -environment interaction-reward-trainingresourceslarge, largemulti-trainingdeployment

Challenge: long-trajectory RL training requires system stability and robust generated outputs

Challenge: high-concurrency tools, state consistency and isolation

- GPU-CPU cooperative acceleration**
- interaction-feedback-trainingdeployment
 - CPU acceleration and scaling
 - Heterogeneous compute resources

- Sandbox/containerised tool deployment**
- Connect to MCP
 - Stable, elastic and scalable
 - Same training and deployment environment

throughput efficiency

Challenge: tool calls take a long time, making long-tail bubbles worse

Challenge: multi-turn interaction worsens pipeline bubbles and sharply reduces cluster utilisation

Challenge: multi-turn, multi-level and diverse rewards lengthen and complicate the reward stage

- Multi-step async**
- Current max: 4 steps
 - Need to support 10 steps

- Dynamic workload system**
- Task-pipeline orchestration
 - Elastic compute resources
 - Load balancing

agentimplication:

Async-training RL algorithms: reduce accuracy loss from async execution and increase system async depth to achieve 10x-100x throughput

System-friendly model architecture: inference-compute friendly model scale/MoE sparsity; long-context memory-friendly sparse/linear/hybrid attention

Overall trend in agent inference: more compute, more memory, longer latency, elastic scheduling

- Agent-inference efficiency directly affects RL throughput and deployed-agent ROI/user experience. Full-stack software-hardware optimisation is needed for agent workloads.

More compute: long trajectories x many tools x many agents sharply increase inference-resource demand

- Anthropic: single-agent and multi-agent systems consume 4x and 15x more tokens than chatbots, respectively
- multi-environment interaction; multi-tool deployment

Prediction: inference compute scale could grow 1000x
Compute-utilisation challenge: how to orchestrate subtasks to avoid bubbles, sense load in real time, and schedule the most suitable compute resources

More memory: multi-turn long trajectories x personalisation expand KV cache

- Multi-turn long trajectories: agents and tools are stateful, and task quality depends on long memory
- Agent personalisation means memory cost grows with interaction duration

Prediction: memory demand will keep rising and constrain ultra-long-task performance

Memory optimisation challenge: how to use pooled memory, improve prefix-cache hits, and optimise/compress KV cache

Longer latency: tool I/O creates a new bottleneck

- Combining fast/slow thinking with tool calls increases end-to-end latency
- Agent-interaction+ - environment interaction- interaction high, latency low

Prediction: low-latency machine-machine interaction conflicts with longer tool-call latency

End-to-end latency challenge: how to run inference and tools asynchronously in parallel; parallelise multiple tools; and balance LLM inference latency against tool-call latency

Elastic scheduling: dynamic task orchestration and reliable inference

- Subtasks in long tasks have different priorities
- Agent tasks vary widely, dynamically calling many agents/tools/environments, so compute-resource needs fluctuate and require elastic scaling
- The inference system must be stable and recover quickly

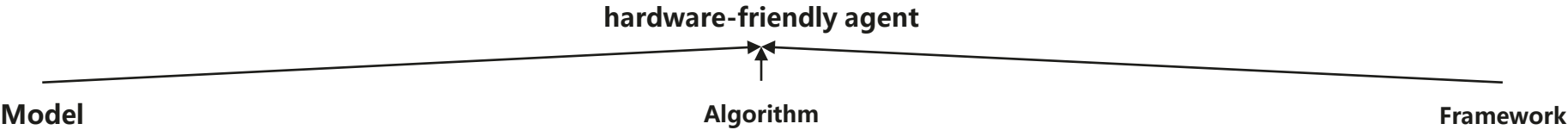
Prediction: systems must dynamically orchestrate many agents/tools and elastic compute resources to meet SLOs

Challenge: how to accelerate CPUs, dynamically tune CPU:GPU ratios, and coordinate GPU-CPU execution

Implications for agent optimisation:

Learn when to generate versus use tools: output efficiently based on task needs and choose the better cost, latency and price-performance path
Parallel and speculative tool-calling algorithms reduce latency: borrow from speculative decoding, predict tool and LLM inference load, and speculatively use tools

computechallengefeedbackModelAlgorithm, agent



- 1. Efficient inference, compute-friendly: medium-scale models and high-sparsity MoE models suited for inference scaling
- 2. Long-context, memory-friendly: linear or hybrid attention such as Lightning Attention

- 1. Async-training RL algorithms reduce accuracy loss: Meta LlamaRL AIPO and Ant AREAL async PPO
- 2. speculative and parallel tool-calling algorithms reduce latency by borrowing from speculative decoding

- 1. Use the same environment for training and production deployment: OpenAI Codex learns in real environments and builds smaller, better-tested modules
- 2. Learn when to generate versus use tools: choose the better cost, latency and price-performance path by scenario

inference-token cost 15x

Anthropic: agentmulti-agentsystemtokenchatmulti-415

Challenge: inference compute demand could grow 1000x, requiring larger inference clusters

High-throughput RL needs inference-interaction-training decoupling and async execution

- RL accuracy declines as data becomes stale
- currently supports at most 4 async steps

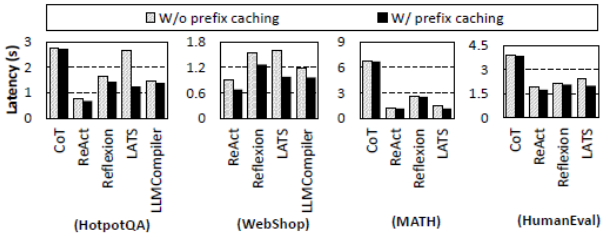
Challenge: long latency from multimodal generation and tool calls requires more async depth, around 10 steps

Agents need isolated and stateful environments

- environments can scale independently

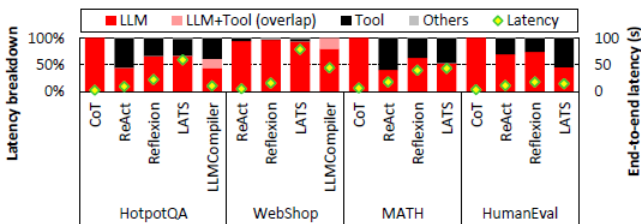
Challenge: Environment compute must remain stable and preserve output robustness

accumulated long context causes high latency



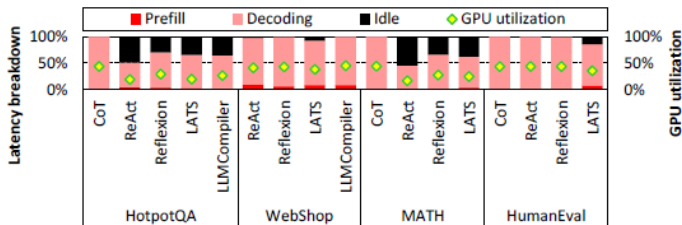
Challenge: repeated trajectories become the next-round context, creating large KV-cache cost

Tool calls materially increase end-to-end latency



Challenge: tool-call I/O latency can account for up to 50%

Inference generation-bound + tool I/O-bound



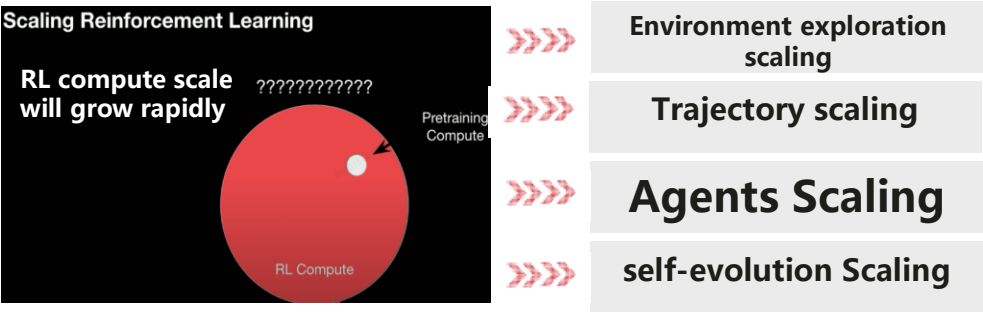
Challenge: serial inference and tool execution -> GPU idle time

Agentic RL training: open environments, long-trajectory learning and self-evolution for next-generation models

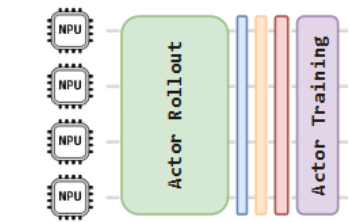
all depend on Agentic RL infrastructure

RL training becomes more complex with agentic tool use: rollout generation takes longer and has heavier tail variance; as RL compute scales faster, improving compute utilisation becomes mandatory

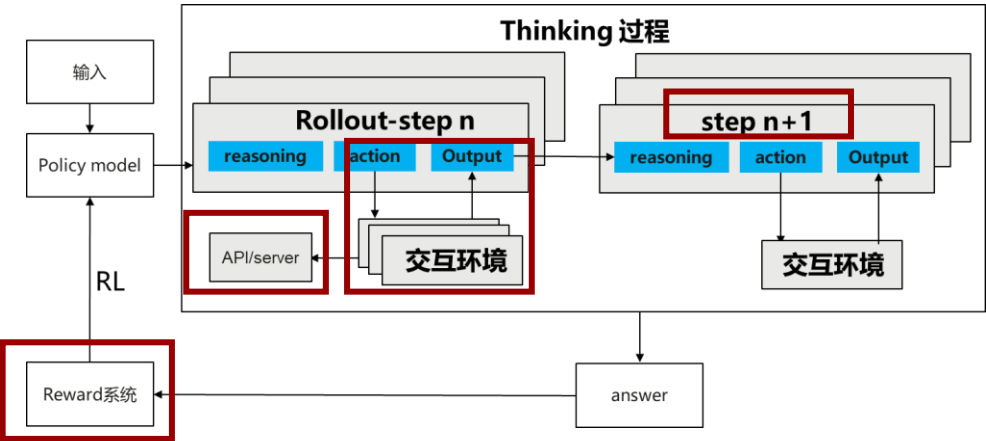
Agentic RL is a prerequisite for training next-generation models, and RL scale will continue to expand



challenge2: -environment interaction-reward-trainingresourceslarge, largemulti-trainingdeployment

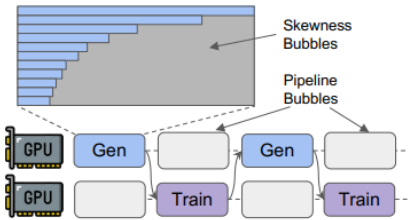


Agentic RL adds multi-modal, multi-environment, multi-tool, multi-turn, multi-agent and multi-reward characteristics



Examples: ByteDance Retool, Microsoft agentic tool use, Nemotron-Research-Tool-N1

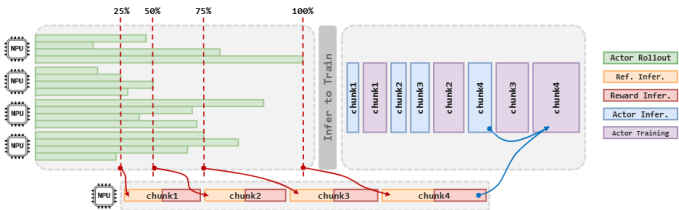
challenge3: tool, multi-environment interactionlong-tailbubbles, pipelinebubbles= =largelow



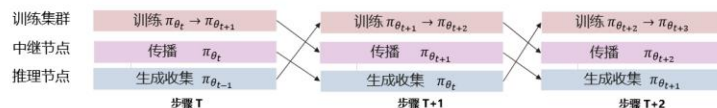
Challenge 4: reward is no longer a single forward pass after one turn; multi-turn, multi-level and diverse rewards lengthen the reward stage and increase scheduling complexity

S0	c	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	X11	X12	X13
Hidden State	Task Context	Agent output				Env. Output				Agent Output				
Per-token PPO														
Per-turn PPO														
Per-trajectory PPO														

Challenge 5: explore more async steps; current systems support around 4 quasi-async steps, and the impact of async depth on accuracy remains unclear



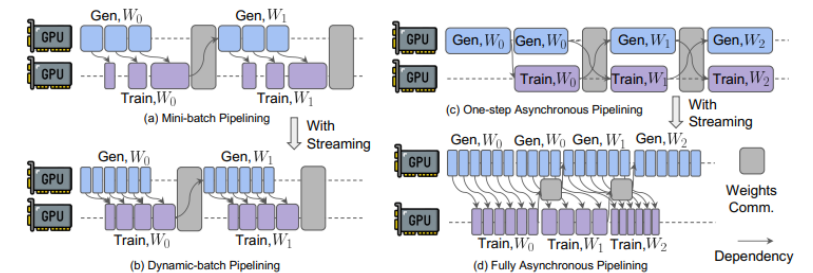
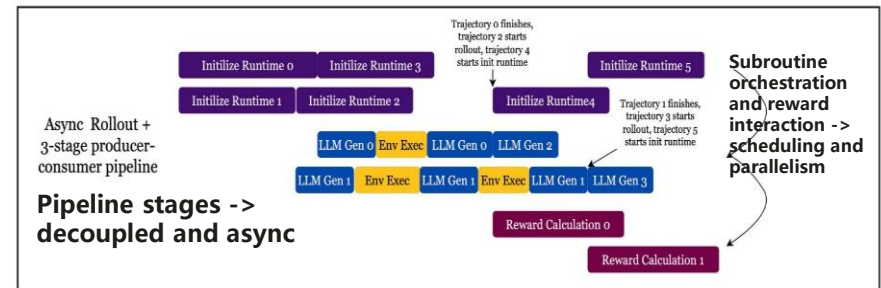
Agentic AI infrastructure needs several new capabilities; industry R&D should accelerate in 2H25, so preparation should start immediately



- Staleness-aware training
- Supports inconsistent policy versions
- Decouples PPOs requirement that training data in one batch comes from the same policy by using π_i behav to account for different policy versions

$$J(\theta) = \mathbb{E}_{q \sim \mathcal{D}, a_t \sim \pi_{\text{behav}}} \left[\sum_{t=1}^H \min \left(\underbrace{\frac{\pi_{\theta}}{\pi_{\text{behav}}}}_{\text{Importance Ratio}}, \hat{A}_t, \overbrace{\frac{\pi_{\text{prox}}}{\pi_{\text{behav}}} \text{clip} \left(\frac{\pi_{\theta}}{\pi_{\text{prox}}}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t}^{\text{Importance Ratio}} \right) \right]$$

$$= \mathbb{E}_{q \sim \mathcal{D}, a_t \sim \pi_{\text{behav}}} \left[\sum_{t=1}^H \frac{\pi_{\text{prox}}}{\pi_{\text{behav}}} \min \left(u_t^{\text{prox}}(\theta) \hat{A}_t, \text{clip} \left(u_t^{\text{prox}}(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right],$$



Agent long-task optimisation: environment compute becomes a critical layer for both training and deployment

- Long-trajectory RL and environment compute will support agent deployment in more real business scenarios

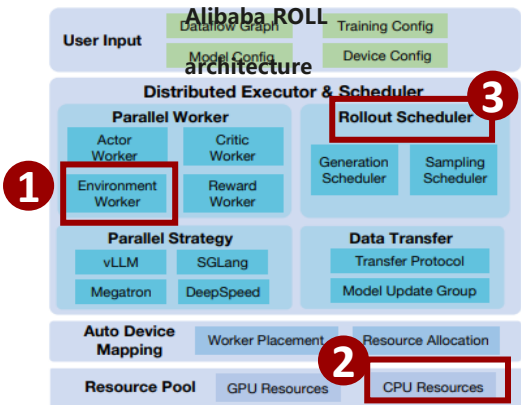
RL architecture must be upgraded for long-task environment compute

Challenge 1: environment resource consumption is huge. In SWE-Bench, each container needs >1 CPU and around 7GB of storage. With batch size 16 and 8 concurrent rollouts, this may require over 100 CPUs and nearly 1TB of disk space.

Challenge 2: CPU-side environments and tools are slow, blocking GPUs and reducing system efficiency

Challenge 3: async parallel environment workers; rollout scheduler breaks traditional batch-processing limits, manages at single-sample granularity and supports priority sample scheduling

challenge4: system stable, : agents small, Environment compute must remain stable and preserve output robustness



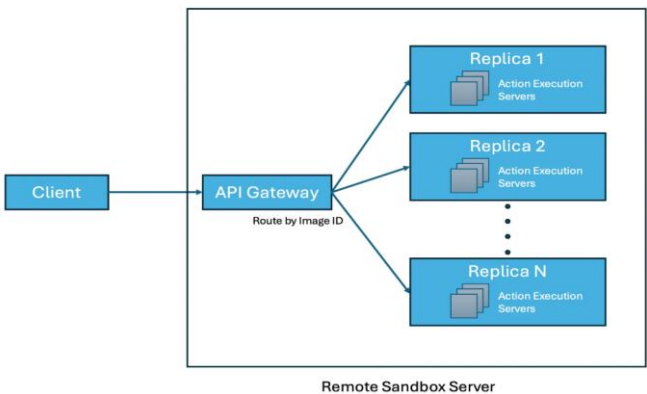
Agents need isolated and stateful environments

OpenAI Codex: real learning environments:

- Training and production deployment use the same environment and the same containerised infrastructure

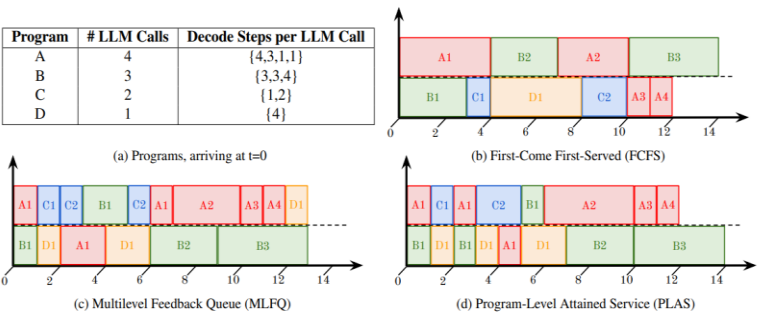
- Build smaller, better-tested modules

SkyRL: scalable environment worker: , environment training, environments can scale independently, from high-frequency environment interaction, rollout stage GPU resource high.



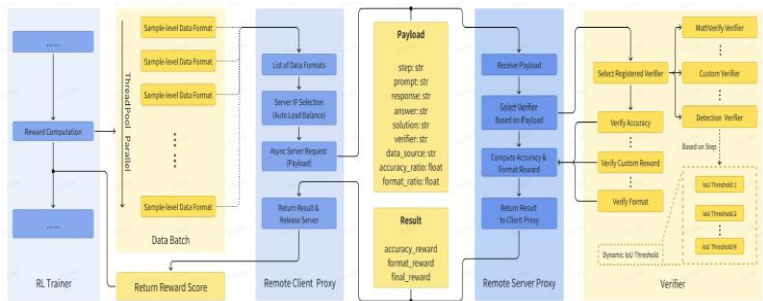
orchestration environment interaction efficiency

A task contains multiple subtasks with different rollout lengths and counts, requiring different orchestration optimisation strategies



25.05 MiniMax Vriune async

The RL trainer interacts with different verifier types in reward servers and returns diverse rewards asynchronously by subtask



Overall trend in agent inference: more compute, more memory, longer latency, elastic scheduling

	reasoningModel	AgenticModel ~task
compute	Test time scaling-compute	multi-highcompute; multi-Modeldeployment; multi-tooldeployment
memory	cache	multi-turn, stateful, tasklong memory
	deploymentModel	agentMCP, A2A; -interaction+ -environment interactionnew
scheduling	deploymentModelscheduling	dynamictool/; multi-agentscheduling; highlatencySLO;

Inference systems should be designed for agents and let agents sense system resources to manage them more efficiently and self-optimize

More compute: inference-token cost 15x

Anthropic blog: single-agent and multi-agent systems consume 4x and 15x more tokens than chat, respectively

tokens fast. In our data, agents typically use about 4× more tokens than chat interactions, and multi-agent systems use about 15× more tokens than chats. For economic viability, multi-agent systems

Change: inference demand could grow 1000x, requiring larger inference clusters

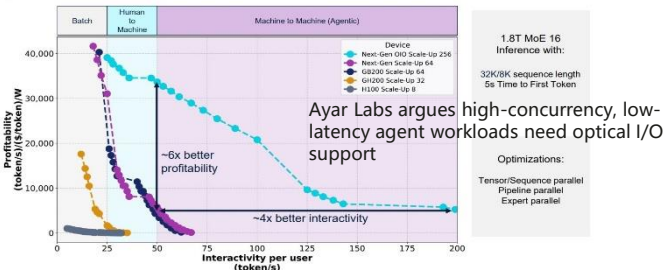
More memory: multi-turn long sequences expand KV cache

- Agent-native memory and stateful tools both carry memory cost
- Long-task agent performance depends on long memory; longer interaction time requires efficient cache management and multi-tier storage
- Agent personalisation means memory cost keeps rising; with many agents, memory may grow super-linearly

Change: memory demand will keep rising; pooled memory is needed

Longer latency: tool calls extend latency

- Combining fast/slow thinking with tool calls increases end-to-end latency
- As agents work independently for longer, MCP and A2A protocol traffic may surge: machine-machine plus machine-world interactions are more frequent and lower-latency than human-machine interactions

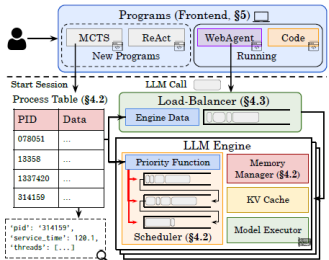


Change: low-latency machine-machine interaction conflicts with longer tool-call

latency

Elastic scheduling: dynamic task orchestration and reliable inference

- Must support priority scheduling across subtasks in long tasks
- The inference system must be stable and recover quickly: agenttask
- Agent tasks vary widely; compute-resource needs fluctuate and require elastic scaling



Change: dynamically orchestrate models/tools and elastically schedule compute resources to keep the

system efficient

